

B.TECH 1ST YEAR · 1ST SEMESTER · BEEE

Digital Electronics

UNIT III — Formula Sheet

Last-Day Revision Weapon · Exam-Ready Reference · Complete Concept Sheet

7

MAJOR TOPICS

8

LOGIC GATES

15+

TRUTH TABLES

30

FLASHCARDS

10+

EXAM TIPS

NUMBER SYSTEM COMPARISON

SYSTEM	BASE	DIGITS USED	PREFIX	EXAMPLE
Decimal	10	0-9	none	45 ₁₀
Binary	2	0, 1	0b	101101 ₂
Octal	8	0-7	0o	55 ₈
Hexadecimal	16	0-9, A-F	0x	2D ₁₆

CONVERSION METHODS

Decimal → Binary: Repeated division by 2, read remainder bottom-up

$$\begin{aligned} 45 \div 2 &= 22 \text{ r } 1 & 22 \div 2 &= 11 \text{ r } 0 & 11 \div 2 &= 5 \text{ r } 1 \\ 5 \div 2 &= 2 \text{ r } 1 & 2 \div 2 &= 1 \text{ r } 0 & 1 \div 2 &= 0 \text{ r } 1 \end{aligned}$$

Binary → Decimal: Positional weight sum

$$\begin{aligned} 101101_2 &= \\ 1 \times 2^5 &+ 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = \\ 32 &+ 0 + 8 + 4 + 0 + 1 = 45_{10} \end{aligned}$$

OCTAL & HEX SHORTCUTS

Binary ↔ Octal: Group binary into sets of 3 bits from right

$$101 \ 101_2 \rightarrow 5 \ 5_8 \rightarrow 55_8$$

Binary ↔ Hex: Group binary into sets of 4 bits from right

$$0010 \ 1101_2 \rightarrow 2 \ D_{16} \rightarrow 2D_{16}$$

Hex ↔ Decimal:

$$2D_{16} = 2 \times 16^1 + 13 \times 16^0 = 32 + 13 = 45_{10}$$

BINARY ARITHMETIC RULES

Addition Rules:

$$\begin{aligned} 0 + 0 &= 0 & 0 + 1 &= 1 \\ 1 + 0 &= 1 & 1 + 1 &= 10 \text{ (carry 1)} \end{aligned}$$

1's Complement:

Flip all bits 0 → 1, 1 → 0
1's comp of 0101 = 1010

2's Complement:

2's comp = 1's comp + 1
0101 → 1010 + 1 = 1011

Subtraction (A-B):

$A - B = A + (2\text{'s comp of } B)$
Discard carry if any

DECIMAL · BINARY · OCTAL · HEXADECIMAL REFERENCE

DEC	BINARY	OCT	HEX	DEC	BINARY	OCT	HEX
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D

6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F



02 / Coding Systems

BCD · EXCESS-3 · GRAY · HAMMING

BCD – 8421 CODE

- ▶ 4 bits represent one decimal digit (0–9)
- ▶ Weights: 8, 4, 2, 1
- ▶ Only 10 of 16 combinations valid
- ▶ Invalid: 1010 to 1111

45_{10} in BCD = 0100 0101

EXCESS-3 CODE

- ▶ BCD + 3 (add 0011 to each BCD digit)
- ▶ Self-complementing code
- ▶ 9's complement = bit complement

Excess-3 of digit D = BCD(D) + 0011
 $45_{10} \rightarrow \text{BCD}(4)+3, \text{BCD}(5)+3 = 0111\ 1000$

BCD VS EXCESS-3 VS GRAY CODE — COMPARISON

DECIMAL	BCD (8421)	EXCESS-3	GRAY CODE
0	0000	0011	0000
1	0001	0100	0001
2	0010	0101	0011
3	0011	0110	0010
4	0100	0111	0110
5	0101	1000	0111
6	0110	1001	0101
7	0111	1010	0100
8	1000	1011	1100
9	1001	1100	1101

GRAY CODE CONVERSION

Binary → Gray:

- ▶ MSB of Gray = MSB of Binary
- ▶ Each subsequent Gray bit = XOR of adjacent Binary bits

$$\begin{aligned} G_1 &= B_1 \text{ (MSB)} & G_2 &= B_1 \oplus B_2 & G_3 &= B_2 \oplus B_3 \\ G_4 &= B_3 \oplus B_4 \end{aligned}$$

Gray → Binary:

$$\begin{aligned} B_1 &= G_1 \text{ (MSB)} & B_2 &= B_1 \oplus G_2 & B_3 &= B_2 \oplus G_3 \\ B_4 &= B_3 \oplus G_4 \end{aligned}$$

Key Property: Only **1 bit changes** between successive values

HAMMING CODE (ERROR DETECTION/CORRECTION)

- ▶ Adds **parity bits** at positions $2^0, 2^1, 2^2, 2^3 \dots$
- ▶ Can detect and correct **1-bit errors**
- ▶ Can detect **2-bit errors** (with extra parity)

No. of parity bits (p): $2^p \geq p + m + 1$ where m = data bits
For m=4: $2^p \geq p+5 \rightarrow p = 3$ (min parity bits)

Parity bit positions: 1, 2, 4, 8, 16...

Error detection: Check all parity bits → Syndrome word points to error position



03 / Logic Gates

BASIC · UNIVERSAL · SPECIAL PURPOSE

BASIC GATE

AND

$$Y = A \cdot B$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Output HIGH only when ALL inputs are HIGH

BASIC GATE

OR

$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Output HIGH when ANY input is HIGH

BASIC GATE / INVERTER

NOT

$$Y = A'$$

A	Y
0	1
1	0

Output is complement of input. 1 input only

DRIVER / BUFFER

BUFFER

$$Y = A$$

A	Y
0	0
1	1

Output same as input, strengthens signal

UNIVERSAL GATE ★

NAND

$$Y = (A \cdot B)'$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Complement of AND. Can implement ANY gate

UNIVERSAL GATE ★

NOR

$$Y = (A + B)'$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Complement of OR. Can implement ANY gate

SPECIAL GATE

XOR

$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

HIGH when inputs are **DIFFERENT**

SPECIAL GATE

XNOR

$$Y = (A \oplus B)'$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

HIGH when inputs are **SAME**. Equality checker

UNIVERSAL GATE: NAND IMPLEMENTATIONS

NOT using NAND:

$$A' = (A \cdot A)' = \text{NAND}(A, A)$$

AND using NAND:

$$A \cdot B = ((A \cdot B)')' = \text{NAND}(\text{NAND}(A, B), \text{NAND}(A, B))$$

OR using NAND:

$$A + B = (A' \cdot B')' = \text{NAND}(\text{NAND}(A, A), \text{NAND}(B, B))$$



04 / Boolean Algebra

LAWS · THEOREMS · DE MORGAN · SIMPLIFICATION

BOOLEAN LAWS & THEOREMS — COMPLETE REFERENCE

LAW / THEOREM	AND FORM	OR FORM
Identity Law	$A \cdot 1 = A$	$A + 0 = A$
Null Law	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent Law	$A \cdot A = A$	$A + A = A$
Complement Law	$A \cdot A' = 0$	$A + A' = 1$
Commutative Law	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative Law	$A(BC) = (AB)C$	$A+(B+C)=(A+B)+C$
Distributive Law	$A(B+C) = AB+AC$	$A+BC = (A+B)(A+C)$
Absorption Law	$A(A+B) = A$	$A + AB = A$
Double Complement	$(A')' = A$	
Involution	$A'' = A$	

★ DE MORGAN'S THEOREM — FIRST

$$(A + B)' = A' \cdot B'$$

The complement of a sum = product of complements

NOR gate = AND gate with inverted inputs

★ DE MORGAN'S THEOREM — SECOND

$$(A \cdot B)' = A' + B'$$

The complement of a product = sum of complements

NAND gate = OR gate with inverted inputs

SOP & POS FORMS

SOP (Sum of Products): OR of AND terms

$$F = A'B + AB' + AB \text{ (each term is a minterm)}$$

POS (Product of Sums): AND of OR terms

$$F = (A+B)(A'+B')(A+B') \text{ (each term is a maxterm)}$$

Minterm: all variables present in AND term

Maxterm: all variables present in OR term

BOOLEAN SIMPLIFICATION EXAMPLE

Simplify: $F = AB + AB'$

$$\begin{aligned} F &= AB + AB' = A(B + B') \\ [\text{Distributive}] &= A(1) \quad [\text{Complement: } B+B'=1] \\ &= A \quad [\text{Identity: } A \cdot 1 = A] \end{aligned}$$

Simplify: $F = A + A'B$

$$\begin{aligned} F &= A + A'B = (A + A')(A + B) \\ [\text{Distributive}] &= 1 \cdot (A + B) = A + B \end{aligned}$$

05 / Combinational Circuits

HALF ADDER · FULL ADDER · COMPARISON

⦿ Half Adder

Adds **2 single bits**. No carry input.

Inputs: A, B | **Outputs:** Sum (S), Carry (C)

Sum (S) = A ⊕ B (XOR gate)
Carry (C) = A · B (AND gate)

Truth Table:

A	B	SUM (S)	CARRY (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Gates used: 1 XOR + 1 AND

Limitation: Cannot add carry from previous stage

⦿ Full Adder

Adds **3 bits** including carry-in (C_{in}).

Inputs: A, B, C_{in} | **Outputs:** Sum (S), Carry-out (C_{out})

Sum (S) = A ⊕ B ⊕ C_{in}
C_{out} = AB + BC_{in} + AC_{in}

Truth Table:

A	B	C _{in}	SUM (S)	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Gates used: 2 XOR + 3 AND + 1 OR OR 2 Half Adders + 1 OR

Use: Cascaded for multi-bit addition

HALF ADDER VS FULL ADDER — COMPARISON

PARAMETER	HALF ADDER	FULL ADDER
No. of Inputs	2 (A, B)	3 (A, B, C _{in})
No. of Outputs	2 (Sum, Carry)	2 (Sum, C _{out})
Carry Input	Not possible	Yes (C _{in})
Sum Expression	A ⊕ B	A ⊕ B ⊕ C _{in}
Carry Expression	A · B	AB + BC _{in} + AC _{in}
Cascading (multi-bit)	Not directly	Yes, easily cascaded
Circuit Complexity	Simpler	Complex
Gates Required	1 XOR + 1 AND	2 XOR + 2 AND + 1 OR



06 / Sequential Circuits & Flip-Flops

SR · JK · D · T FLIP-FLOPS · REGISTERS · COUNTERS

KEY CONCEPTS

- **Sequential Circuit:** Output depends on current inputs + **past states (memory)**
- **Feedback:** Output fed back as input — stores state
- **Clock:** Controls when state changes occur
- **Synchronous:** All flip-flops change on clock edge
- **Asynchronous:** No common clock signal

FLIP-FLOP OVERVIEW

- Basic memory element storing **1 bit**
- Has 2 stable states: $Q = 0$ or $Q = 1$
- Outputs: Q (normal) and Q' (complement)
- Types: SR, JK, D, T
- Triggered on: Rising edge / Falling edge / Level

SR Flip-Flop (Set-Reset)

Basic flip-flop. S=Set ($Q \rightarrow 1$), R=Reset ($Q \rightarrow 0$)

$$Q(n+1) = S + R'Q(n) \text{ Constraint: } S \cdot R = 0 \text{ (forbidden state)}$$

S	R	$Q(n+1)$	STATE
0	0	$Q(n)$	No Change
0	1	0	Reset
1	0	1	Set
1	1	X	Forbidden !

JK Flip-Flop (Improved SR)

Eliminates forbidden state. J=Set, K=Reset

$$Q(n+1) = JQ'(n) + K'Q(n) \text{ No forbidden state! } JK=1 \rightarrow \text{Toggle}$$

J	K	$Q(n+1)$	STATE
0	0	$Q(n)$	No Change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(n)$	Toggle ✓

D Flip-Flop (Data/Delay)

Stores data bit. Output follows input on clock edge.

$$Q(n+1) = D \text{ (Output = Input, delayed by 1 clock)}$$

D	$Q(n+1)$	STATE
0	0	Reset
1	1	Set

Used in: Registers, data storage, shift registers

T Flip-Flop (Toggle)

Toggles state when $T=1$. No change when $T=0$.

$$Q(n+1) = T \oplus Q(n) \text{ (Toggle when } T=1)$$

T	$Q(n+1)$	STATE
0	$Q(n)$	No Change
1	$Q'(n)$	Toggle

Used in: Frequency dividers, counters

FLIP-FLOP COMPARISON TABLE

TYPE	INPUTS	CHARACTERISTIC EQUATION	FORBIDDEN STATE	KEY APPLICATION
SR	S, R	$Q(n+1) = S + R'Q$	$SR = 1$	Basic memory
JK	J, K	$Q(n+1) = JQ' + K'Q$	None	Counters, registers
D	D	$Q(n+1) = D$	None	Data storage

T

T

$$Q(n+1) = T \oplus Q$$

None

Frequency dividers

REGISTERS

- ▶ Group of flip-flops storing multi-bit data
- ▶ N flip-flops → N-bit register
- ▶ **SISO:** Serial In Serial Out
- ▶ **SIPO:** Serial In Parallel Out
- ▶ **PISO:** Parallel In Serial Out
- ▶ **PIPO:** Parallel In Parallel Out
- ▶ Shift register: data shifted left/right on each clock

COUNTERS

- ▶ Sequential circuit that counts clock pulses
- ▶ **Ripple Counter:** Async, output of one FF = clock of next
- ▶ **Synchronous Counter:** All FFs clocked simultaneously
- ▶ N-bit counter: can count 0 to 2^N-1
- ▶ MOD-N counter: cycles through N states
- ▶ **UP counter:** Counts 0,1,2... 2^N-1 ,0...
- ▶ **DOWN counter:** Counts 2^N-1 ...1,0, 2^N-1 ...

N-bit counter: 2^N states MOD-N: N
clock pulses per cycle



07 / Quick Flashcards

30 EXAM-ORIENTED Q&A CARDS

Q: Base of Binary number system?

A: 2 (uses digits 0 and 1)

Q: Universal gates?

A: NAND and NOR

Q: XOR output condition?

A: HIGH when inputs are DIFFERENT ($A \neq B$)

Q: De Morgan's 1st theorem?

A: $(A+B)' = A' \cdot B'$

Q: De Morgan's 2nd theorem?

A: $(A \cdot B)' = A' + B'$

Q: Half adder outputs?

A: Sum ($S = A \oplus B$) and Carry ($C = AB$)

Q: Full adder inputs?

A: 3 inputs – A, B, Carry-in (C_{in})

Q: 2's complement of a number?

A: 1's complement + 1

Q: Gray code property?

A: Only 1 bit changes between successive codes

Q: BCD code weights?

A: 8, 4, 2, 1 (8421 code)

Q: Excess-3 = ?

A: BCD + 0011 (add 3 to BCD digit)

Q: Forbidden state in SR FF?

A: $S=1, R=1$ (both HIGH simultaneously)

Q: JK FF when $J=1, K=1$?

A: Toggle (Q changes to Q')

Q: D Flip-Flop characteristic equation?

A: $Q(n+1) = D$

Q: T Flip-Flop characteristic equation?

A: $Q(n+1) = T \oplus Q(n)$

Q: XNOR output condition?

A: HIGH when inputs are SAME ($A=B$)

Q: NOT gate using NAND?

A: $A' = \text{NAND}(A, A)$

Q: OR gate using NAND?

A: $\text{NAND}(\text{NAND}(A, A), \text{NAND}(B, B))$

Q: Binary → Octal shortcut?

A: Group bits in sets of 3 from right

Q: Binary → Hex shortcut?

A: Group bits in sets of 4 from right

Q: Idempotent Law?

A: $A \cdot A = A$ and $A + A = A$

Q: Complement Law (AND)?

A: $A \cdot A' = 0$

Q: Complement Law (OR)?

A: $A + A' = 1$

Q: Number of parity bits for 4 data bits?

A: 3 parity bits (Hamming code)

Q: N-bit counter can count up to?

A: 2^N states (0 to 2^N-1)

Q: Hex digit A = ? in decimal?

A: 10

Q: Hex digit F = ? in decimal?

A: 15

Q: What is a Ripple Counter?

A: Async counter – output of each FF clocks the next

Q: SOP stands for?

A: Sum of Products (OR of AND terms)

Q: What does D FF use in circuits?

A: Data storage, shift registers, memory elements



08 / Exam Tips & Common Mistakes

AVOID THESE ERRORS TO SCORE FULL MARKS

⚠ MISTAKE #1 — NUMBER CONVERSIONS

- ▶ Always read binary remainders **bottom-up** when converting decimal→binary
- ▶ Don't forget to group from **right side** for Binary↔Octal/Hex
- ▶ Add leading zeros to complete groups (e.g., 101 → 000 101 for hex grouping)

⚠ MISTAKE #2 — DE MORGAN'S THEOREM

- ▶ Never write $(A+B)' = A'+B'$ ← WRONG!
- ▶ Correct: $(A+B)' = A' \cdot B'$ and $(AB)' = A'+B'$
- ▶ Remember: complement flips operator AND complements variables

⚠ MISTAKE #3 — TRUTH TABLES

- ▶ n-input gate → always write 2^n rows
- ▶ Fill inputs in binary counting order (00, 01, 10, 11...)
- ▶ Don't mix up XOR (different) vs XNOR (same)
- ▶ NAND is NOT the same as NOT AND drawn separately

⚠ MISTAKE #4 — NAND VS NOR LOGIC

- ▶ NAND = $(AB)'$ — invert after AND
- ▶ NOR = $(A+B)'$ — invert after OR
- ▶ Don't apply De Morgan randomly — show step-by-step

⚠ MISTAKE #5 — ADDER OUTPUTS

- ▶ Half Adder Sum = $A \oplus B$ (NOT $A + B$ or AB)
- ▶ Full Adder Sum includes carry-in: $A \oplus B \oplus C_{in}$
- ▶ Always write both Sum AND Carry expressions
- ▶ Full Adder Carry = $AB + BC_{in} + AC_{in}$ (3 terms!)

⚠ MISTAKE #6 — FLIP-FLOP OPERATIONS

- ▶ SR FF: $S=R=1$ is FORBIDDEN — always state this
- ▶ JK FF has NO forbidden state — key advantage over SR
- ▶ D FF output always equals input D (no complex equation)
- ▶ $T=1$ in T FF means TOGGLE (not set to 1!)
- ▶ Write $Q(n+1)$ notation, not just Q

⚠ MISTAKE #7 — GRAY CODE

- ▶ MSB of Gray = MSB of Binary (unchanged)
- ▶ For Gray→Binary: XOR with PREVIOUS BINARY bit (not Gray)
- ▶ Don't confuse Gray and BCD — they're completely different

✅ PRO TIPS FOR FULL MARKS

- ▶ Always draw the logic diagram when asked
- ▶ Write Boolean expression before drawing
- ▶ Show conversion steps — don't just write final answer
- ▶ Mention gate count when comparing circuits
- ▶ State characteristic equation for every flip-flop



09 / Quick Revision Sheet

ALL KEY FORMULAS & EXPRESSIONS AT A GLANCE

🔢 NUMBER SYSTEMS

Binary: Base 2 | Octal: Base 8
Hex: Base 16 | Decimal: Base 10
Dec→Bin: divide by 2, read remainders ↑
Bin→Oct: group 3 bits (right to left)
Bin→Hex: group 4 bits (right to left)
1's comp = flip all bits
2's comp = 1's comp + 1
 $A-B = A + 2\text{'s comp}(B)$

🔢 CODES

BCD weights: 8-4-2-1
Excess-3 = BCD + 0011
Gray MSB = Binary MSB
Bin→Gray: $G_1=B_1$, $G_n=B_{n-1} \oplus B_n$
Gray→Bin: $B_1=G_1$, $B_n=B_{n-1} \oplus G_n$
Hamming: $2^p \geq p + m + 1$
Parity bit positions: 1,2,4,8,16...
Gray: only 1 bit changes at a time

⚙️ LOGIC GATES

AND: $Y = A \cdot B$
OR: $Y = A + B$
NOT: $Y = A'$
NAND: $Y = (AB)'$ [Universal]
NOR: $Y = (A+B)'$ [Universal]
XOR: $Y = A \oplus B$ (inputs differ)
XNOR: $Y = (A \oplus B)'$ (inputs same)
NOT via NAND: $A' = \text{NAND}(A, A)$

📐 BOOLEAN LAWS

$A \cdot 1 = A$ | $A + 0 = A$ (Identity)
 $A \cdot 0 = 0$ | $A + 1 = 1$ (Null)
 $A \cdot A = A$ | $A + A = A$ (Idempotent)
 $A \cdot A' = 0$ | $A + A' = 1$ (Complement)
 $A'' = A$ (Double Complement)
 $A + AB = A$ (Absorption)
 $(A+B)' = A'B'$ (De Morgan 1)
 $(AB)' = A' + B'$ (De Morgan 2)

🔧 COMBINATIONAL CIRCUITS

Half Adder: $S = A \oplus B$, $C = AB$
Full Adder: $S = A \oplus B \oplus C_{in}$
Full Adder $C_{out} = AB + BC_{in} + AC_{in}$
HA inputs: 2 | FA inputs: 3
HA gates: 1 XOR + 1 AND
FA gates: 2 XOR + 2 AND + 1 OR
FA = 2 Half Adders + 1 OR gate
SOP=OR of AND | POS=AND of OR

🔄 FLIP-FLOPS

SR: $Q(n+1) = S + R'Q$ | $SR = 0$
SR forbidden: $S=R=1$
JK: $Q(n+1) = JQ' + K'Q$
JK: $J=K=1 \rightarrow \text{Toggle}$
D: $Q(n+1) = D$
T: $Q(n+1) = T \oplus Q$
 $T=1 \rightarrow \text{Toggle}$ | $T=0 \rightarrow \text{No change}$
N-bit counter: 2^N states

★ LAST-MINUTE MEMORY ANCHORS ★

Gate Memory Trick:

- ▶ AND = All must be 1 → output 1
- ▶ OR = Any 1 → output 1
- ▶ NAND = AND flipped
- ▶ NOR = OR flipped
- ▶ XOR = eXclusive = different
- ▶ XNOR = Same = equality

Flip-Flop Memory Trick:

- ▶ SR = Set/Reset (with forbidden)
- ▶ JK = Just like SR but smarter
- ▶ D = Delay (Q follows D)
- ▶ T = Toggle (flips when T=1)
- ▶ JK best for counters
- ▶ D best for storage/registers

Code Memory Trick:

- ▶ BCD = Binary for each decimal digit
- ▶ Excess-3 = BCD + 3
- ▶ Gray = Cyclic, only 1 bit changes
- ▶ Hamming = Error correction
- ▶ NAND/NOR = Universal (any circuit)
- ▶ XOR = Core of adders